Page: Details | Result: 1081 - 7474 - kernel | Add Baseline | Apply Rules | Occupancy Calculator | Source Comparison | Save as PDF

	Result	Time	Cycles	GPU	SM Frequency	Process	Attributes
Current	7474 - kernel (72, 1, 4)x(16, 4, 1)	5.63 usecond	9,008	0 - NVIDIA GeForce RTX 4060 Laptop GPU	1.60 cycle/nsecond	[11435] run	

GPU Speed Of Light Throughput

GPU Throughput Rooflines

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	19.19	Duration [usecond]	5.63
Memory Throughput [%]	28.01	Elapsed Cycles [cycle]	9,008
L1/TEX Cache Throughput [%]	25.18	SM Active Cycles [cycle]	6,863.25
L2 Cache Throughput [%]	7.39	SM Frequency [cycle/nsecond]	1.60
DRAM Throughput [%]	28.01	DRAM Frequency [cycle/nsecond]	6.91

Latency Issue This kernel exhibits low compute throughput and memory bandwidth utilization relative to the peak performance of this device. Achieved compute throughput and/or memory bandwidth below 60.0% of peak typically indicate latency issues. Look at Scheduler Statistics and Warp State Statistics for potential reasons.

Roofline Analysis The ratio of peak float (fp32) to double (fp64) performance on this device is 64:1. The kernel achieved close to 0% of this device's fp32 peak performance and 0% of its fp64 peak performance. See the Kernel Profiling Guide for more details on roofline analysis.

PM Sampling

Timeline view of PM metrics sampled periodically over the workload duration. Data is collected across multiple passes. Use this section to understand how workload behavior changes over its runtime.

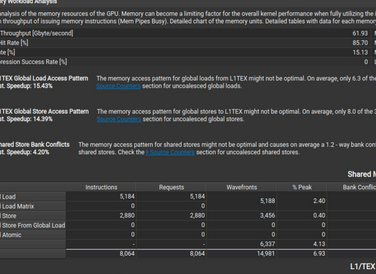
Maximum Sampling Interval [usecond]	2	# Pass Groups	2
Maximum Buffer Size [Mbytes]	4	Dropped Samples [sample]	0

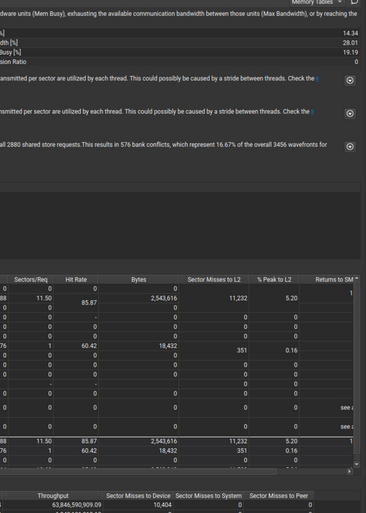
Compute Workload Analysis

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

Executed Ipc Elapsed [inst/cycle]	0.69	SM Busy [%]	23.28
Executed Ipc Active [inst/cycle]	0.90	Issue Slots Busy [%]	23.28
Issued Ipc Active [inst/cycle]	0.93		

Balanced *ALU* is the highest-utilized pipeline (22.6%) based on active cycles, taking into account the rates of its different instructions. It executes integer and logic operations. It is well-utilized, but should not be a bottleneck.





Memory Workload Analysis

Memory Tables

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

Memory Throughput [Gbyte/second]	61.93	Mem Busy [%]	14.34
L1/TEX Hit Rate [%]	85.70	Max Bandwidth [%]	28.01
L2 Hit Rate [%]	15.13	Mem Pipes Busy [%]	19.19
L2 Compression Success Rate [%]	0	L2 Compression Ratio	0

L1TEX Global Load Access Pattern Est. Speedup: 15.43% The memory access pattern for global loads from L1TEX might not be optimal. On average, only 6.3 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Source Counters section for uncoalesced global loads.

L1TEX Global Store Access Pattern Est. Speedup: 14.39% The memory access pattern for global stores to L1TEX might not be optimal. On average, only 8.0 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Source Counters section for uncoalesced global stores.

Shared Store Bank Conflicts Est. Speedup: 4.20% The memory access pattern for shared stores might not be optimal and causes on average a 1.2 - way bank conflict across all 2880 shared store requests.This results in 576 bank conflicts, which represent 16.67% of the overall 3456 wavefronts for shared stores. Check the Source Counters section for uncoalesced shared stores.

Shared Memory

	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	5,184	5,184	5,188	2.40	0
Shared Load Matrix	0	0			
Shared Store	2,880	2,880	3,456	0.40	576
Shared Store From Global Load	0	0	0	0	0
Shared Atomic	0	0	0	0	0
Other			6,397	4.13	0
Total	8,064	8,064	14,981	6.93	576

L1/TEX Cache

	Instructions	Requests	Wavefronts	% Peak	Sectors	Sectors/Req	Hit Rate	Bytes	Sector Misses to L2	% Peak to L2	Returns to SM
Local Load	0	0	0	0	0	0	0	0	0		0
Global Load	6,912	6,912			79,488	11.50	85.87	2,543,616	11,232	5.20	1
Global Load To Shared Store (access)	0	0	10,870	5.03	0	0	-	0	0	0	
Global Load To Shared Store (bypass)	0	0			0	0	-	0	0	0	
Surface Load	0	0	0	0	0	0	0	0	0	0	
Texture Load	0	0	0	0	0	0	0	0	0	0	
Global Store	576	576	576	0.27	576	1	60.42	18,432	351	0.16	
Local Store	0	0	0	0	0	0	0	0		0	
Surface Store	0	0	0	0	0	0	0	0	0	0	
Global Reduction	0	0	0	0	0	0	0	0	0	0	
DSMEM Reduction									0	0	
Surface Reduction	0	0	0	0	0	0	0	0	0	0	
Global Atomic ALU	0	0	0	0	0	0	0	0	0	0	see
Global Atomic CAS	0	0									
Surface Atomic ALU	0	0	0	0	0	0	0	0	0	0	see
Surface Atomic CAS	0										
Loads	6,912	6,912	10,870	5.03	79,488	11.50	85.87	2,543,616	11,232	5.20	1
Stores	576	576	576	0.27	576	1	60.42	18,432	351	0.16	
Atomics & Reductions	0	0	0	0	0	0	0	0	0	0	

L2 Cache

	Requests	Sectors	Sectors/Req	% Peak	Hit Rate	Bytes	Throughput	Sector Misses to Device	Sector Misses to System	Sector Misses to Peer
L1/TEX Load	3,744	11,237	3.00	3.98	7.37	359,584	63,846,590,909.09	10,404	0	0
L1/TEX Store	317	342	1.08	0.24	100	10,944	1,943,181,818.18	0	0	0
L1/TEX Atomic ALU	0	0	0	0	0	0	0	0	0	0
L1/TEX Atomic CAS	0	0	0	0	0	0	0	0	0	0
L1/TEX Reduction	0	0	0	0	0	0	0	0	0	0
L1/TEX Total	4,075	11,572	2.84	4.10	10.21	370,304	65,790,000,000	10,404	0	0
ECC Total	-	0	-	0	-	0	0	0	-	-
GPU Total	4,389	13,494	3.07	4.78	16.40	431,808	76,670,454,545.45	10,425	0	0

L2 Cache Eviction Policies

	First	Hit Rate	Last	Hit Rate	Normal	Hit Rate	Normal Demote	Hit Rate
L1/TEX Load	10,373	0.05	0	0	855	95.79	0	0
L1/TEX Store	0	0	0	0	340	100	0	0
L1/TEX Atomic	0	0	0	0	0	0	-	-
L1/TEX Total	10,368	0	0	0	1,219	97.05	0	0
GPU Total	10,642	2.57	0	0	1,379	97.32	0	0

Device Memory

	Sectors	% Peak	Bytes	Throughput
Load	10,900	28.01	348,800	61,931,818,181.82
Store	0	0	0	0.24
Total	10,900	28.01	348,800	61,931,818,181.82

Scheduler Statistics

Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]	2.77	No Eligible [%]	75.54
Eligible Warps Per Scheduler [warp]	0.31	One or More Eligible [%]	24.46
Issued Warp Per Scheduler	0.24		

Issue Slot Utilization Est. Local Speedup: 71.99% Every scheduler is capable of issuing one instruction per cycle, but for this kernel each scheduler only issues an instruction every 4.1 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 12 warps per scheduler, this kernel allocates an average of 2.77 active warps per scheduler, but only an average of 0.31 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, avoid possible load imbalances due to highly different execution durations per warp. Reducing stalls indicated on the Warp State Statistics and Scheduling Statistics sections can help, too.

Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined values.

Warp Cycles Per Issued Instruction [cycle]	11.33	Avg. Active Threads Per Warp	23.93
Warp Cycles Per Executed Instruction [cycle]	11.70	Avg. Not Predicated Off Threads Per Warp	20.04

Long Scoreboard Stalls Est. Speedup: 34.68% On average, each warp of this kernel spends 3.9 cycles being stalled waiting for a scoreboard dependency on a L1TEX (local, global, surface, texture) operation. Find the instruction producing the data being waited upon to identify the culprit. To reduce the number of cycles waiting on L1TEX data accesses verify the memory access patterns are optimal for the target architecture, attempt to increase cache hit rates by increasing data locality (coalescing), or by changing the cache configuration. Consider moving frequently used data to shared memory. This stall type represents about 34.7% of the total average of 11.3 cycles between issuing two instructions.

Warp Stall Check the Warp Stall Sampling (All Samples) table for the top stall locations in your source based on sampling data. The Kernel Profiling Guide provides more details on each stall reason.

Thread Divergence Est. Speedup: 7.17% Instructions are executed in warps, which are groups of 32 threads. Optimal instruction throughput is achieved if all 32 threads of a warp execute the same instruction. The chosen launch configuration, early thread completion, and divergent flow control can significantly lower the number of active threads in a warp per cycle. This kernel achieves an average of 23.9 threads being active per cycle. This is further reduced to 20.0 threads per warp due to predication. The compiler may use predication to avoid an actual branch. Instead, all instructions are scheduled, but a per-thread condition code or predicate controls which threads execute the instructions. Try to avoid different execution paths within a warp when possible.

Instruction Statistics

Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that 'Instructions/Opcode' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions [inst]	148,608	Avg. Executed Instructions Per Scheduler [inst]	1,548
Issued Instructions [inst]	153,408	Avg. Issued Instructions Per Scheduler [inst]	1,598

FP32 Non-Fused Instructions Est. Speedup: 1.11% This kernel executes 4608 fused and 3456 non-fused FP32 instructions. By converting pairs of non-fused instructions to their fused, higher-throughput equivalent, the achieved FP32 performance could be increased by up to 21% (relative to its current performance). Check the Source page to identify where this kernel executes FP32 instructions.

NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.

NVLink Tables

Detailed tables with properties for each NVLink.

NUMA Affinity

Non-uniform memory access (NUMA) affinities based on compute and memory distances for all GPUs.

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	288	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	166	Static Shared Memory Per Block [byte/block]	0
Block Size	64	Dynamic Shared Memory Per Block [byte/block]	272
Threads [thread]	18,432	Driver Shared Memory Per Block [kbyte/block]	1.02
Waves Per SM	2	Shared Memory Configuration Size [kbyte]	16.38
Uses Green Context	0	# SMs [SM]	24

Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	25	Block Limit Registers [block]	6
Theoretical Active Warps per SM [warp]	12	Block Limit Shared Mem [block]	11
Achieved Occupancy [%]	21.58	Block Limit Warps [block]	24
Achieved Active Warps Per SM [warp]	10.36	Block Limit SM [block]	24

Theoretical Occupancy Est. Speedup: 71.99% The 3.00 theoretical warps per scheduler this kernel can issue according to its occupancy are below the hardware maximum of 12. This kernel's theoretical occupancy (25.0%) is limited by the number of required registers.

GPU and Memory Workload Distribution

Analysis of workload distribution in active cycles of SM, SMP, SMSP, L1 & L2 caches, and DRAM

Average SM Active Cycles [cycle]	6,863.25	Average L1 Active Cycles [cycle]	6,863.25
Average L2 Active Cycles [cycle]	4,067	Average SMSP Active Cycles [cycle]	6,532.92
Average DRAM Active Cycles [cycle]	10,900	Total SM Elapsed Cycles [cycle]	216,160
Total L1 Elapsed Cycles [cycle]	216,160	Total L2 Elapsed Cycles [cycle]	141,088
Total SMSP Elapsed Cycles [cycle]	864,640	Total DRAM Elapsed Cycles [cycle]	155,648

Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	16,704	Branch Efficiency [%]	83.33
Branch Instructions Ratio [%]	0.11	Avg. Divergent Branches	18

Uncoalesced Global Accesses Est. Speedup: 31.85% This kernel has uncoalesced global accesses resulting in a total of 55296 excessive sectors (69% of the total 80064 sectors). Check the L2 Theoretical Sectors Global Excessive table for the primary source locations. The CUDA Programming Guide has additional information on reducing uncoalesced device memory accesses.

L2 Theoretical Sectors Global Excessive

Location	Value	Value (%)
0x7a63f2544490 in kernel	9,064	16
0x7a63f2544510 in kernel	9,064	16
0x7a63f2542e0 in kernel	9,064	16
0x7a63f2542a0 in kernel	9,064	16
0x7a63f2542450 in kernel	9,064	16

Uncoalesced Shared Accesses Est. Speedup: 5.08% This kernel has uncoalesced shared accesses resulting in a total of 576 excessive wavefronts (7% of the total 8640 wavefronts). Check the L1 Wavefronts Shared Excessive table for the primary source locations. The CUDA Best Practices Guide has an example on optimizing shared memory accesses.

L1 Wavefronts Shared Excessive

Location	Value	Value (%)
0x7a63f2544770 in kernel	576	100
0x7a63f2544ea0 in kernel	0	0
0x7a63f2544fd0 in kernel	0	0
0x7a63f2544f00 in kernel	0	0
0x7a63f2544ff0 in kernel	0	0

Follow the rules outputs to get guidance on how to navigate through the report and quickly discover performance bottlenecks in this kernel.
You could also disable Collection Actions to focus on selected performance aspects and make profiling faster.